AI Report

We are not confident this text is

AI Generated

AI Probability

48%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism

?

The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

first_draft-2.pdf - 4/12/2026

Charles O Malley

Enhancing Model Checker Test Generation with Functional Programming

Charles O'Malley, BAI

A Thesis

Presented to the University of Dublin, Trinity College

in partial fulfilment of the requirements for the degree of

Master of Science in Computer Science (Computer Engineering)

Supervisor: Andrew Butterfield

April 2026

Enhancing Model Checker Test Generation with Functional Programming

Charles O'Malley, Master of Science in Computer ScienceUniversity of Dublin, Trinity College, 2026

Supervisor: Andrew Butterfield

This dissertation explores whether the refinement stage of an existing SPIN/Promelabased RTEMS test-generation pipeline can be moved from Python to Haskell without changing the behaviour of the generated tests.

The work focuses on refine command, a stateful part of the system that turns annotated SPIN traces into C test code.
Instead of replacing the whole pipeline at once, the project uses a bridge-based approach so that the Haskell version can be introduced gradually and checked against the original Python implementation.
The port eventually moved away from line-by-line translation and towards a more Haskell-native design based on explicit state and smaller transformations.
The two versions were then compared across the seven active models in the repository.
This process found one real bug in the Haskell output, traced to template brace handling, which was fixed.
After that, the remaining differences were limited to harmless formatting.
Overall, the project shows that the refiner can be ported in a behaviour-preserving way.

Acknowledgments

Thank you Mum \& Dad.

Charles O'Malley

University of Dublin, Trinity College

April 2026

Contents

List of Tables

List of Figures

Chapter 1

Introduction

This chapter introduces the dissertation, motivates the decision to port part of the RTEMS test-generation pipeline from Python to Haskell, and explains the structure of the document.
It first describes the technical context of the project, then states the aim and scope of the work, and finally shows how the remaining chapters support the overall argument.

1.1 Motivation

The repository used in this project contains a SPIN/Promela-based workflow for generating RTEMS validation tests.
Promela models are explored with SPIN, the resulting annotated traces are refined into concrete test actions, and those actions are emitted as C test files.
The stage studied here is the refinement step, implemented historically in Python.
That step is small enough to replace in isolation, but central enough that any change to it can affect the entire pipeline.

The motivation for the project came from the structure of the existing refiner rather than from a failure of the generated tests.
The Python version already generated useful RTEMS tests, but it accumulated mutable state, branching logic, and template expansion in one place.
That made it harder to reason about the code, and it made larger structural changes risky.
Haskell was therefore investigated not as a way to generate different tests, but as a way to re-express the same refinement logic with clearer state handling and stronger separation between stages.

In addition i was interested in exploring the coding paradigm that is fucntional programming, as i had nto had the oppurtunity to investigate it before due to the modules covering it not being avaible to me as a computer engineer. self teaching myself haskell would be a great way to explore it deeply.

1.2 Aim and Scope

The main aim of the dissertation is to determine whether the refine_command stage can be ported from Python to Haskell while preserving the behaviour of the existing generator.
In practical terms, that means the Haskell version should accept the same annotated SPIN output and produce equivalent test code for the existing RTEMS models.

The scope of the work is deliberately narrow.
The dissertation does not attempt to redesign the full testbuilder workflow, replace RTEMS itself, or prove that Haskell is universally better than Python for test generation.
Instead, it focuses on one difficult, stateful part of the toolchain and studies how a gradual port can be carried out without losing confidence in the output.

In order to smoothly port the python program to haskell, the dissertation aims to create a incremental delegation strategy to keep end to end functionality consistent while the program is being created.
the idea is to have an intermediary program that sits in between the file to be ported and the file that calls it.
the original implementation will be preseved in situe with the new implementation.
at the beggining the intermediary program will give all work to the original implementation, but as the porting continues, the middleware will delegate more and more work to the new implementation until it is complete.
i came up with this appraoch due to the desire to be entirely confident with the output of the system at all points in the project as i knew that the tests for rtems, the system which this codebase is for, are checked by rigourous quality control specialists.
if i were to check the results of a partial program, i would want to be sure that it was progressing positively but i couldnt necessarily be sure of that just by looking at it myself.
So i figured however that if the program in the end produced the same output as teh original that it would be a satisfactory port and i wouldnt have to know the details of the model checking or test generation to eb sure of it.
so the aim to put this idea into practice

The main contributions of the dissertation are:

devising a bridge-based strategy for introducing large code changes to an existing codebase without interupting end to end functionality

an explicit-state Haskell implementation of the refinement stage;

an empirical comparison of Python and Haskell generated outputs across the seven active Promela models in the repository.

1.3 Dissertation Structure

Chapter 2 reviews the background needed for the project, including model-checker-based test generation, functional programming, and multilingual system design.
Chapter 3 defines the problem more precisely and explains the design decisions behind the bridgebased port.
Chapter 4 describes how the refiner was moved from Python to Haskell and where the implementation changed from line-by-line translation to a more Haskell-native design.
Chapter 5 evaluates the result by comparing generated outputs and discussing the remaining limitations of the workflow.
Chapter 6 closes the dissertation and identifies the most useful directions for further work.

Chapter 2

State of the Art

This chapter looks at the disserations place within the background work that made it project possible.
It first reviews model checker based test generation, then discusses the functional programming ideas that motivate its use and the specific choice of Haskell, and finally considers the practical problems of combining Python and Haskell inside one toolchain.

2.1 Model Checking and Test Generation

Model checking is normally presented as a verification technique, but in this project it is used in a more different way.
SPIN explores the Promela model, produces counterexample traces.
Traces are sequences of events, in this case recorded when assertions are violated, and those traces are then treated as the input for a test code generator rather than as the final result.
The important point for this dissertation is that the generated test cases do not come directly from the model checker alone.
They depend on a second translation stage that maps the traces onto RTEMS-oriented test code.

the pipeline is therefore a two stage system, which is incomplete with model checking alone.
In the second step the traces produced by Spin are intepreted into test code using rtems specific refinement rules and C test code fragments and then assembled into a complete test file.
the correctness of teh test produced therefore are dependent not just on the searching of the model checker but of the quality of the translation layer after it.

That distinction between the first and second stage of the test generation matters for this disseration because the dissertation does not replace the modelling or exploration stages.
The Promela models, SPIN search, and RTEMS test structure already existed and remained untouched for this project.
The difficult engineering problem of concern is the refinement step that comes after the promela mode and spin traces but before completed test code: reading annotation lines such as CALL, STATE, STRUCT, and SEQ,

tracking enough context to interpret them correctly, and emitting the right C fragments in the right order to create an executable that can verify teh behaviour of a real RTEMs deployment.

this translation layer is not simply a thin middleware that renames tokens with a fixed template where every line is intepreted separately to the whole.
each line requires additional context from previous lines and must continue the context to the following lines.
It must be able to determine whether the lines have entered or left higher level constructs such as strucutres or sequences.
The translation layer can not be applied to any new system with manual adjustement first due to the fact that there are refinement dcitionaries and templates that are specific to each model that are required for the operation of the translation layer.
For these reasons the refiner is better viewed as a stateful transformation system than a thin formatting layer.

the work of Micskei and Majzik (Micskei and Majzik, 2006) comes closer to how the pipeline as seen in this dissertation exists.

They use model checkers to create tests for embedded systems, showing that model checkers have value in actual systems and not just in theory as had been the case to that point.
Ulrich

A broader example of the same general idea appears in the work of Zheng et al.
Zheng et al 2007.
They use model checking to derive tests for service behaviour described in BPEL-style models.
The domain is different from RTEMS, and the modelling formalism is different from the Promela workflow used here, but the wider pattern is similar: model checking is used to systematically derive execution scenarios, and a second stage turns those scenarios into executable tests.
That makes this dissertation easier to place in a broader test-generation tradition rather than treating it as an isolated RTEMS-specific toolchain.

et al.
(Ulrich et al., 2010) are also relvant in that they emphasise the implementation step that comes after the model checker search.
Ulrich wrote the program to create tests in Java but eventually came to decouple the backend from java in order to support other output langauges.
This extensibility was something i wanted to emulate in this project

In the context of model checking and test generation for RTEMS, the output of a model checker test generator must be valid C code that fits the functionality and quality standards of the exisitn test suites generated by hand by the RTEMS team.
this standard was achieved by my predecessor who worked on the whole of the test generator system as applied to RTEMS (Jaśkuć, 2022) applied model checker test generation to RTEMS in the first place and thus is extremely relevant to this dissertation as its the code that is actually being converted.
The entire system is written in Python as it was mandated by authority in control of the repository.
this led to the compromise that they programs were written in python depsite the developers desire to be writitg in something fucntional,

which ended up in the use of the external library Coconut which extends ptyhon to be more functional.
why they would want to make it more fucntional is explored further in the next seciont

Seen in the light of this existing paper, this is not another demonstration that model checking can support automasted test generation, this has already been illustrated well by the paper.
this dissertation instead is about whther the existing trace to code refinment system can be reimplemented without intterujpting or damaging the behaviour of the overall generation process.
this changed focus makes the disseration closer to a study of a behaviour rpeserving codebase refactoring then to the work whos main contribution is the creation of a new modelling test generation framework.
the model checking test generation is just the area of application that is being studied in this disseration.

2.2 Functional Programming and Haskell

A useful starting point for the Haskell side of the project is Hughes's argument that modularity is one of the strongest practical benefits of functional programming (Hughes, 1989).

This is directly relevant here because the refiner is a transformation: it reads one representation of behaviour and incrementally builds another.
A style that makes intermediate state and data flow explicit is therefore attractive, especially when behaviour must be preserved while the internal structure changes.

That argument is useful here mainly because the refiner is full of small, state-dependent transformations.
The code does not need a language chosen for theoretical but usually unrealised benefits; it needs a language whose normal style encourages clearer composition and more local reasoning.
In that sense the appeal of functional programming in this dissertation is practical rather than abstract.
The main benefit is not that the program becomes mathematically elegant, but that the steps by which one annotation line changes the next runtime state can be expressed more directly and thus more intuitively altered.

This project invovled learning about the history design goals of Haskell, summarised by Hudak et al.
Hudak et al 2007.
Haskell's combination of strong static typing, algebraic data types, pattern matching, and purity by default makes it well suited to programs that perform structured translation.
THis is because, at least for this specific transformation there is constant pattern matching and also confusion that can arise from typing as traces which are ouputted as strings are necessarily processed, extracting the integers values held within.
These properties do not automatically make the code correct , but they encourage an implementation style in which cases are more explicit and state changes are easier to trace.

That history is also relevant because it frames Haskell as a language designed to sup-

port reusable abstraction and a clear common standard, rather than as a niche experiment.
For this dissertation that matters because the port is not trying to prove that Haskell is universally better than Python.
It is simply using a language whose usual tools fit a translation problem well: explicit data modelling, case analysis, and a clearer separation between pure transformation and boundary-facing input and output.

It is also worth talking about the scope of haskells utilisation.
The haskell as implemented in this project was learned for the purpose of this dissertation rather than from an extended period of getting to sit with the language in a formal academic module.
This matters in that the port being done by the author will not be able to fully use all the theoretical benefits of the language to its fullest extent.
Even so, with just a subset of the languages potential, the refinement program can be improved and then it will greatly ease further contributors in maximising the programs potential

2 3 Multi Language Systems

The project is also part of a broader class of multilingual systems, where one language is introduced into an existing codebase rather than replacing it completely.
A recent empirical study by Yang et al.
(Yang et al., 2024) highlights interface boundaries, data conversion, and error diagnosis as recurring sources of friction.

Those concerns match the experience of this project closely.
The main difficulty was not writing isolated Haskell functions, but deciding how Python and Haskell should communicate without making the combined system more fragile than the original one.

This problem shaped the eventual architecture.
Instead of a deep foreign-function interface, the project used a narrow subprocess protocol over standard input and output.
That choice keeps the boundary explicit.
Configuration is passed in as JSON, annotation lines are refined one at a time, and the rendered test body is returned at the end.
The approach is less elegant than a direct import, but it is easier to inspect, easier to debug, and better aligned with the existing Python pipeline.
It also reflects the main lesson of the multilingual-systems literature in a fairly direct way: once the boundary is explicit, it becomes easier to reason about what data crosses it and where failures should be diagnosed.

the later design and implementation chapters unpack this in more concrete terms by lookijg at the trade offs in inspectability, debugabillity and API stability.

Two concrete integration options were considered during the project.
The HaPy library (Fisher, 2014) suggested a more direct Python-to-Haskell interface by exposing compiled Haskell code to Python, but its documented setup depends on shared-library builds and a limited set of supported exported types.
The call-haskell-from-anything li-

brary (Hambüchen, 2011) explored a more general MessagePack-based boundary over the C foreign-function interface.
Both were useful as engineering reference points, but neither was adopted.
The repository needed a controlled way to preserve the existing Python API while changing the implementation behind it, and the simpler subprocess boundary turned out to be the safer fit for that requirement.

2.4 Closely Related Work

The most directly relevant prior work is the earlier dissertation by Jaśkuć (Jaśkuć, 2022).
That work is important because it establishes the value of the underlying RTEMS/SPIN framework itself.
The present dissertation does not need to justify that the entire modelling approach is worthwhile; it builds on an existing system that had already shown that Promela models and SPIN traces could be refined into useful RTEMS tests.

The difference in emphasis is therefore clear.
The earlier work extended the range and usefulness of the RTEMS modelling framework.
This dissertation instead studies the maintainability of one important part of that framework by replacing the refinement stage with a Haskell version.
The contribution is not a new manager model or a new coverage result, but a controlled port of the translation logic used by the existing models.

| Source | Main Idea | Relevance Here |

|

| Why Functional Programming Matters (Hughes, 1989) | Functional programming improves modularity and composition | Motivates the move from implicit mutation toward explicit transformations |

| A History of Haskell (Hudak et al., 2007) | Explains the design goals of Haskell as a common non-strict functional language | Justifies Haskell as a plausible replacement language for a structured translation stage |

| Yang et al.
(Yang et al., 2024) | Identifies interface boundaries and data conversion as common developer difficulties | Motivates the explicit subprocess bridge between Python and Haskell |

| Fraser et al.
(Fraser et al., 2009) | Surveys how model checkers are used to derive test cases from counterexamples and witnesses | Clarifies that this dissertation focuses on the later refinement step that makes those traces usable |

| Micskei and Majzik (Micskei and Majzik, 2006) | Generates tests for event-driven embedded systems using model checkers | Shows that model-checkerdriven test generation is relevant in embedded, reactive software settings |

| Ulrich et al.
(Ulrich et al., 2010) | Moves from formal scenarios to concrete test implementations via Promela | Reinforces that a non-trivial implementation step exists between formal scenarios and runnable tests |

| Zheng et al.
(Zheng et al., 2007) | Uses model checking to derive executable tests from behavioural models | Shows that turning modelchecker output into tests is a broader pattern, not just an RTEMS-specific one |

| Staroletov (Staroletov, 2020) | Models partitioned real-time operatingsystem behaviour in Promela for SPINbased verification | Provides systems-software context for using Promela and SPIN on operatingsystem behaviour |

| Jaśkuć (Jaśkuć, 2022) | Demonstrates the value of the RTEMS/SPIN testgeneration framework | Provides the immediate technical context that this dissertation extends |

In summary, the literature and engineering context suggest two main conclusions.
First, the project looks to be a study of how model-checker output can be preserved while the refinement stage is restructured.
Second, the challenge is not only one of programming language choice, but also one of managing a clear interface between old and new implementations.

Chapter 3

Design

This chapter turns the background of the previous chapter into concrete design decisions.
It defines the specific problem within the existing RTEMS toolchain, identifies the constraints that any replacement must satisfy, and explains the bridge-based design used to introduce Haskell gradually rather than by one large swap.

3.1 Problem Formulation

The problem addressed by this dissertation is not that the existing generator failed to produce useful RTEMS tests.
The problem is that one central stage of the pipeline, the Python refiner, became difficult to restructure safely as the project grew.
The testbuilder workflow already had working Promela models, a working SPIN-based exploration stage, and working C test generation.
Changing the refiner therefore meant changing the part of the system where the risk of regression was highest.

The existing Python refiner combines several kinds of work in one place.
It parses annotation lines, tracks context such as whether refinement is currently inside a STRUCT or SEQ, performs lookups in a refinement dictionary, and accumulates generated code fragments.
This arrangement works, but it makes it harder to reason locally about how one line of input affects the next state of the translation.

Three properties of the original implementation were especially important.
First, context is stored as mutable object state.
Second, the main refinement function handles many annotation forms in one place.
Third, template expansion and code accumulation are interleaved with control flow.
Each of these is manageable in isolation, but together they make structural changes riskier than they need to be.

3.1.1 Identified Challenges

The first challenge was behavioural preservation.
The repository already contained working models and a working Python refiner.
A replacement could only be justified if it produced the same observable output for the existing model set.

The second challenge was integration.
The refiner is not a standalone tool.
It is called from the existing Python pipeline and depends on YAML refinement dictionaries, language templates, and per-model header fragments.
Any design therefore had to fit around those existing inputs rather than invent a new workflow.

The third challenge was scope control.
Replacing the entire pipeline at once would have made it difficult to distinguish language-porting issues from unrelated tooling issues.

A narrower design was needed so that the hardest stateful component could be rewritten first and checked continuously against the original behaviour.

3.1.2 Proposed Work

The design adopted in this dissertation is a staged port of refine_command.
The Haskell implementation keeps the same external role as the Python one: it receives tokenised annotation lines, updates its refinement state, and eventually renders a test body.
The surrounding Python code is retained so that existing configuration files, model assets, and generation scripts can continue to drive the process.

The central design decision is therefore not only to use Haskell, but to introduce it behind a bridge that preserves the historical Python-facing interface.
That keeps the migration incremental.
The new implementation can be checked with the same inputs as the old one, and output differences can be detected before they spread into the rest of the workflow.

the bridge was placed at the boundary between spin2test.py and refinecommand.py for good reason.
when spin2test.py hands work to refinecommand.py, most of the surrounding setup has already been done, the correct RTEMS model files have been found, the .spn ouput has been split into individual spin trace lines nad the YAML refinement dictionaries specific to the RTEMS model have already been loaded.
This leaves refinecommand.py with a more singular and specialised job.
it only has to recieve configuration settings at the beginning, process the spin trace lines one by one, and then return the completed test scripts.
putthing the bridge here isolates the stateful, pattern matching part of the pipeline while keeping the function of the ported file more consolidated by not also having it handle other parts of the workflow.
this design decision complements the decision to use the bridge to gradually port refinecommand.py.
this is because when comparing the output of the new file with the original, if theres a difference

then i can be more sure that it has to do with the test conversion pipeline and not with some auxiliary helper function.

Two more aggressive and involved alternatives were considered for this bridge program but were ultimately not chosen.
Firstly a full rewrite at the outset would have moved the bridge location outwards, meaning that more of the system could be interfaced with using haskell, but it would also have linked refine_command.py changes with spin2test.py, file orchestration and template loading.
as mentioned before this would have made regressions trickier to pinpoint, having a wider area to check.

Secondly, a Foreign Function Interface (FFI) is another appraoch to the mediating between the haskell and non haskell portions of the code, this time having haskell be called as a compiled library from haskell, like the equivalent of a native library.
this way keeps the refinment inside of one process, as opposed to have the bridge process and the refinement sub process.

It wasnt chosen however as it creates a more brittle boundary where things like function signatures, data layout and memory ownership have to be decided exactly.
additonally it makes debugging less transparent as the haskell function is called directly by the python code.
the stdin/out message pipeline contrasts this by being quite flexible with its command response system and being extremely visible for debugging, as every message is published to console.
these reasons explain why this appraoch was chosen

3.2 Overview of the Design

The overall design separates the system into three logical parts: SPIN produces annotated traces from Promela models, the bridge passes those annotations and configuration data to the Haskell refiner, and the existing Python-side tooling assembles the final test files around the rendered body.
This decomposition isolates the port to the refinement stage while leaving the rest of the pipeline stable.

In the repository as it stands now the direction of control remains Python centric.
spin2test.py reads the generated Spin trace file, keeps only lines that start with @@@, splits those lines into pieces for the test code templates, loads the refinement dictionary from YAML, and constructs the command object.
That object name is preserved so the programs that call for it do not have to change, but the implementation behind it is now the bridge in combinedhaskellbridge.py.
The bridge starts the Haskell executable in interactive mode and copies the methods that the existing Python pipeline expects, including setupLanguage, refineSPINLinemain, and testbody.

A line-oriented protocol was used between Python and Haskell because it made the interface explicit and easy to inspect.
The bridge sends an initial JSON configuration,

then a sequence of refinement commands, and finally requests the rendered test body.
The protocol is simple enough to log and replay, which proved useful during debugging.

INIT JSON:{...}

RESPONSE:SUCCESS:INIT JSON received

COMMAND setupLanguage

RESPONSE SUCCESS setupLanguage

COMMAND refineSPINLine 4 CALL WaitForSuspend 2 resumeRC]

RESPONSE SUCCESS refineSPINLine

COMMAND testBody

RESPONSE SUCCESS testBody BEGIN

.

RESPONSE SUCCESS testBody END

Listing 3.1: Simplified bridge exchange between Python and Haskell

Here is a concrete example to help show why this appraoch works.
In the task-manager model, spin2test.py can encounter the spin line ["4", "CALL", "WaitForSuspend", "2", "resumeRC"] while scanning the generated trace.
reducing every part of the trace line into a separate piece.
The token array is sent over the bridge, and the Haskell refiner then performs the stateful work.
It begins by recording process ID 4, then applying any switch or annotation handling required by the runtime flags, in this case its "CALL" which doesnt require any switching, then looking up WaitForSuspend in the refinement dictionary, and formatting the associated template with the arguments 2 and resumeRC.

WaitForSuspend|

Tlog TNORMAL Waiting for Task d to Suspend 0 1;

do{

1 ctx t isSuspend 0?
taskID 0 0xffffffff;

rtemstaskwake after 100;

while 1 RTEMSALREADYSUSPENDED;

Listing 3.2: Refinement template for WaitForSuspend

here is the template that is called up by searching for "WaitForSuspend" in the refinement dictionaries.
the spots where theres number are where the tokens are slotted in

Tlog TNORMAL Waiting for Task d to Suspend 2 resumeRC;

do{

resumeRC ctx t isSuspend 2?
taskID 2 0xffffffff;

rtemstaskwake after 100;

while resumeRC RTEMSALREADYSUSPENDED;

`

Listing 3.3: Generated C after template substitution

and here you can see what it looks like when the values from the spin line are subsituted in.

The important design point is that Python does not need to know how that fragment is built; it only preserves the boundary and passes the same observable inputs that the old refiner used.

The state after that command remains inside the Haskell process.
The generated lines are accumulated in memory as part of the runtime state rather than being written to disk immediately.
Only when Python later calls test_body() does the bridge send COMMAND:testBody, receive the text body between BEGIN and END markers, and then assemble the final output file.
It does this by writing the predefined preamble and postamble text, the rendered body, and the run fragment.
This separation is why the bridge can stay narrow: Python keeps responsibility for file-system orchestration, while Haskell owns the stateful translation from token array stream to generated C.

JSON plus line-oriented commands were sufficient because only two payload shapes had to cross the boundary: one configuration object at startup and one token array per annotation line.
Everything else is a fixed command or a return of the body text.
A richer RPC framework would have added complexity without improving the one property the dissertation needed most, namely a boundary that could be logged, replayed, and compared directly against the historical Python behaviour.

This design also made differential checking a key focus of the project rather than an afterthought.
Because the Haskell refiner could be exercised with the same traces as the Python original, text diffs on generated tests became the main feedback loop.
That in turn made it possible to move from a line-by-line translation strategy to a more Haskell-native design without losing confidence in the observable output.

Against the two main alternatives, the subprocess bridge performed best on the criteria that mattered most for this dissertation.
It preserved the historical Python API because callers still interacted with a familiar command-style object.
It was easier to debug because every request and response could be inspected as plain text.
And it supported debugging checking because the same annotation traces could be replayed through both implementations with minimal surrounding change.
Its main cost was explicit message passing, but that cost was acceptable because the boundary was intentionally narrow.

3.3 Summary

In summary, the design of the dissertation is conservative at the system boundary and experimental inside the refiner itself.

The boundary stays stable so that behaviour can be checked continuously, while the implementation behind that boundary is free to adopt a clearer Haskell structure.

Chapter 4

Implementation

This chapter describes how the design of the previous chapter was realised in the repository.
It concentrates on mechanics rather than motivation: how the Python-facing workflow was preserved, how the bridge starts and talks to the Haskell executable, how diff-based comparison exposed regressions, and how individual annotation lines become generated C.

4.1 Overview of the Solution

the implementation followed a gradual transition from the original Python refiner to a Haskell refiner while keeping the rest of the pipeline recognisable.
In this repository snapshot, the main artefacts were the earlier Python refiner purepythonrefinecommand.py, spin2test.py, combinedhaskellbridge.py, and the new refinecommand.hs.
The objective was not to replace everything at once, but to replace the refinement stage in a way that could be checked end to end against the old behaviour.

A full rewrite from the start would have been risky.
The refiner is small enough to look manageable, but it contains many state-dependent branches and many points where template expansion affects the final test file.
Reaching the end of a full rewrite before comparing outputs would therefore have made it difficult to localise errors.
The staged approach reduced that risk by keeping differential comparison in the loop throughout the port.

The testing and debugging effort was directed toward functional equivalence with the Python generator.
The Python output already matched the expected RTEMS test structure, so the main implementation question was whether the Haskell version could preserve that observable behaviour while using a clearer internal structure.

The conservative part of the implementation was therefore the system boundary.
spin2test.py would still discover the generated trace, load the YAML refinement dictio-

nary, and write the final test file around the rendered body.
The experimental part was the refiner behind that boundary.
This split made it possible to change the most stateful component first while keeping the surrounding workflow stable enough to compare outputs continuously.

4.2 The Implementation

4.2.1 The Bridge

During the transition, a bridge layer preserved the historical Python-facing command interface while moving the refinement work behind a subprocess.
Python remained responsible for orchestrating the wider pipeline, but Haskell could now be exercised inside that pipeline without forcing an immediate rewrite of every caller.

At the Python entry point, the actual calling pattern changed less than the architectural shift might suggest.
spin2test.py still reads the generated .spn file, filters lines beginning with @@@, tokenises those lines, loads the refinement YAML, and writes the final output by placing generated code between the existing preamble, postamble, and run fragments.
The main change is that it now imports combinedhaskellbridge.py as refine and constructs a bridge-backed command object instead of just importing refine_command.py

cmd refine command ref dict)

cmd setupLanguage)

for ln in annote_bundle:

cmd refineSPINLine main ln)

for line in cmd.test_body():

tstfile write line)

Listing 4.1: Python-side calling preserved by the bridge

Inside combinedhaskellbridge.py, the public command wrapper delegates to a smaller _HaskellBackend.
Startup has three visible steps.
First, Python locates the refine-haskell executable and launches it in interactive mode with standard input and output.
Second, it builds one JSON object containing the refinement dictionary and the three runtime flags outputLOG, annoteComments, and outputSWITCH.
These are options that can be turned on and off, outputLOG for examplle toggles whether a log file is created.
Third, it sends that object as an INIT JSON: .
This makes configuration loading explicit and gives one early failure point if JSON parsing or language setup goes wrong.

Communication after startup is carried entirely through standard input and output.
The bridge sends fixed commands such as COMMAND setupLanguage and COMMAND testBody and it sends spin lines as JSON arrays via COMMAND refineSPINLine On the Haskell side handleLine decodes the command prefix updates the in memory runtime state and responds with either RESPONSE SUCCESS or RESPONSE FAILED
On the Python side expect checks that each response begins with the required prefix.
If readline sees an unexpected EOF it triggers an immediate failure.
The result is a deliberately plain protocol in which invalid commands malformed JSON or a crashed subprocess are surfaced quickly at the boundary.

INIT JSON ref dict flags}

RESPONSE SUCCESS INIT JSON received

COMMAND setupLanguage

RESPONSE SUCCESS setupLanguage

COMMAND refineSPINLine 4 CALL WaitForSuspend 2 resumeRC]

RESPONSE SUCCESS refineSPINLine

COMMAND testBody

RESPONSE SUCCESS testBody BEGIN

.

RESPONSE SUCCESS testBody END

Listing 4 2 Short request response exchange used during refinement

After successful initialisation the Haskell executable moves into a simple command loop that continues until standard input reaches EOF.
In practice the process lifetime is therefore tied to the Python caller.
Once spin2test py has requested the final test lines and has exited after writing the generated test file the the standard input output close and the interactive loop terminates.
This keeps the protocol small at the cost of relying on EOF rather than explicit session finalisation.

This arrangement is deliberately plain but that simplicity was useful in practice.
Python continued to own file system orchestration and the historical command interface while Haskell owned only the translation from spin line stream to emitted body text.
That narrow split is what allowed the bridge to be inserted with relatively little change on the Python side.

4 2 2 The Diff Script

A constant part of the porting process was checking the generated test files against the ones produced by the original system.
At first this was done manually with an online text comparison tool but that approach became unworkable once a single change could

affect many generated files.
A small local diff script therefore became part of the working process.

diff for tr task mgr model 16 c-

home a Documents Python gen tr task mgr model 16 c

home a Downloads rtems RTEMS SMP Formal main formal promela models task-

mgr gen tr task mgr model 16 c

177 10 177 10@

Tlog TNORMAL 4 CALL WaitForSuspend 2 resumeRC;

Tlog TNORMAL Waiting for Task d to Suspend 2 resumeRC;

do{

do{

resumeRC ctx t isSuspend 2?
taskID 2 0xffffffff;

rtemstaskwake after 100;

while resumeRC RTEMSALREADYSUSPENDED;

while resumeRC RTEMSALREADYSUSPENDED;

SWITCH 2 Suspension of proc4 in favour of proc5/

SWITCH 3 ReleaseTestSyncSema of proc4 sometime after proc5/

370 6 370 7@

}

Listing 4 3 Example output from the diff workflow showing a generated file diverging from the reference output. In practice even a single extra line was worth investigating because small formatting mismatches could indicate a deeper error in template expansion.

The diff based approach was intentionally strict.
It reported trivial whitespace differences as well as genuinely important mismatches.
That occasionally produced noise but it was still useful because it made it harder for a small divergence to go unnoticed and later turn into a larger behavioural discrepancy.

In practice most debugging started from a diff then moved backwards through the relevant refinement dictionary entry and the code path that emitted it.

The brace mismatch shown in the diff output became a clear example of why this workflow was useful.
In the task manager model the Haskell output initially contained do and while where the Python generator produced a single pair of braces i e. the valid C syntax.
The underlying problem was not in the control flow logic of the generated test but in template handling.
The refinement dictionaries store literal braces doubled so that Python str format will leave them intact.
The first Haskell version replaced

numbered placeholders but did not collapse doubled braces back to single ones afterwards.
The result was syntactically invalid generated C even though the surrounding structure was correct.
Fixing the haskell refiner ended up being as simple as adding an addtional case in teh pattern matching to account for it which restored parity with the Python output.
This was a good example of a bug that was easiest to see at the level of emitted files rather than at the level of isolated helper functions.

Smaller tests were also useful around configuration handling and local translation behaviour but they were less decisive than the end to end diff checks.
The hardest bugs were usually interactions between several moving parts at once lines templates states and rendering.
For those cases comparing the final generated C remained the most informative test.

4 2 3 Refine Command

This was the main file to be converted.
Its job is to take annotated SPIN output and refine it into concrete test code by matching annotation lines against entries in a model specific refinement dictionary.
Those entries are small templates of C code with placeholders for names values and identifiers taken from the trace.

At the beginning of the port the Python code was translated into Haskell as closely as possible.
The result was syntactically invalid generated C even though the surrounding structure was correct.
What became clear quite quickly however was that this strategy made the Haskell version look like Python written in another syntax.
It preserved the awkward parts of the original without gaining much clarity from the new language.

The turning point came in state handling.
The program has to remember whether the current annotation is inside a structure inside a sequence or part of ordinary top level output.
The Python version manages this with mutable object fields that are updated as each line is processed.
That behaviour can be copied in Haskell but copying it directly would have reproduced the same difficulty of following how state changes over time.

The more Haskell native approach was to make the state explicit.

Instead of mutating fields on one long lived object each stage receives a runtime state transforms it and returns the next one.
At that point the port stopped being a line by line translation and became a Haskell implementation that happened to preserve the same outputs.
The aim from then on was not structural similarity to the Python source but behavioural equivalence of the generated tests.

The refiner therefore ended up with a clear two level structure.
One level applies generic steps that every annotation line goes through such as PID collection optional switch insertion and optional annotation logging.
The second level handles the opcode-

specific work such as expanding a CALL template or entering STRUCT SEQ mode.
This separation made it easier to reason about what was common to all lines and what was specific to one kind of annotation.

This change is visible in the following snippets.

Python state stored and mutated on the object

self inSTRUCT False

self inSEQ False

self seqForSEQ"

self inName"

case pid SEQ name:

self inSEQ True

self seqForSEQ"

self inName name

case pid SCALAR value:

self seqForSEQ self seqForSEQ value

Python state stored and mutated on the object

In the Python version the state lives on the command object.
As each SPIN line is read these values are updated in place and later branches depend on whatever earlier branches left behind.

Haskell state made explicit

data RuntimeState RuntimeState

rtInSTRUCT Bool

rtInSEQ Bool

rtSeqForSEQ Text

rtInName Text

rtDefCode Text]

rtDeclCode Text]

rtTestCodes Int Text]

rtCurrentId Int

rtSwitchNo Int

}

Listing 4 5 Explicit runtime state in the Haskell refiner

The same information is still present but it is now grouped into one explicit record.
Rather than hidden mutation on an object each function takes a state and returns the next one.

One SPIN line producing the next state

refineSPINLineMain Text RuntimeState RuntimeState

refineSPINLineMain ln rt0=

let rt1 collectPIdsTokens ln rt0

rt2 if outputSWITCH rtFlags rt1 then switchIfRequired ln rt1 else rt1

rt3 if annoteComments rtFlags rt2 then logSPINLine ln rt2 else rt2

in refineSPINLine ln rt3

Listing 4 6 Processing one SPIN line by returning a new state

This is the clearest expression of the change in approach.
Instead of one long lived object being updated in place each stage produces a fresh version of the runtime state before the next stage runs.

The split between refineSPINLineMain and refineSPINLine also matters conceptually.
The first function handles generic pre processing that can apply to many opcodes.
The second handles the meaning of the opcode itself.
For a CALL annotation for example switch insertion and annotation logging are decided before the dictionary lookup for the named template happens.

CALL refinement via dictionary lookup

pidTxt CALL name args>

case lookupRef name rt of

Nothing addCode pidFromText pidTxt rtEOLC rt CALL no refinement

entry for name rt

Just code>

let callCode if null args then code else formatTemplate code args

in addCode pidFromText pidTxt splitLines callCode rt

Listing 4 7 CALL annotations refined by template lookup

This is one place where the Haskell version became more direct than a literal translation.
The branch says almost exactly what the operation means find the template instantiate it with the supplied arguments split it into output lines and append those lines to the runtime state under the current PID.

Sequence handling as an explicit transition

pidTxt SEQ name>

rt rtInSEQ True rtSeqForSEQ rtInName name}

pidTxt SCALAR value>

rt rtSeqForSEQ rtSeqForSEQ rt value}

Listing 4 8 Sequence state handled as an explicit transition

Sequence handling was where this change felt most useful.
In this context a sequence is an ordered run of values collected between SEQ and END and later emitted through a matching SEQ template.
In the Haskell version entering that mode and accumulating those values are both visible directly in the returned state.

The same explicit state style also makes it easier to explain one full translation example.
In the task manager model the refiner may see an earlier declaration line for resumeRC and then later a CALL annotation that uses the same name inside a generated polling loop.

4 DECL byte resumeRC 0

4 CALL WaitForSuspend 2 resumeRC

WaitForSuspend

Tlog TNORMAL Waiting for Task d to Suspend 0 1;

do{

1 ctx t isSuspend 0?
taskID 0 0xffffffff;

rtemstaskwake after 100;

while 1 RTEMSALREADYSUSPENDED;

Listing 4 9 Annotation and refinement template used in the taskmanager example

The declaration line matters because the later CALL template does not use resumeRC as a literal value.
It uses it as the name of a C variable that must already exist in the generated test.
When the CALL line is processed the runtime state is updated in stages.
First the PID is recorded in rtProcIds.
Second if switch output is enabled and the active PID changes the bridge can emit the corresponding SUSPEND and WAKEUP template expansions.
Third if annotation comments are enabled the raw annotation line is logged into the generated output.
Only after those generic steps does the CALL branch look up WaitFor

Hallucination Scan Report

Citation check (1)

- 0 don't exist online
- 0 may not exist online
- 1 exist online
- 0 are not verifiable

Cited claims (19)

- 0 not supported by source
- 0 partially supported or nuanced
- 2 are supported by source

Uncited claims (3)

- 0 not supported by online sources
- 2 partially supported or nuanced
- 1 are supported by online sources

first_draft-2.pdf

Enhancing Model Checker Test Generation with Functional Programming

Charles O'Malley, BAI

A Thesis

Presented to the University of Dublin, Trinity College
in partial fulfilment of the requirements for the degree of

Master of Science in Computer Science (Computer Engineering)

Supervisor: Andrew Butterfield
April 2026

Enhancing Model Checker Test Generation with Functional Programming

Charles O'Malley, Master of Science in Computer ScienceUniversity of Dublin, Trinity College, 2026

Supervisor: Andrew Butterfield

This dissertation explores whether the refinement stage of an existing SPIN/Promelabased RTEMS test-generation pipeline can be moved from Python to Haskell without changing the behaviour of the generated tests. The work focuses on refine command, a stateful part of the system that turns annotated

Legend

- Citation not found online
- Citation may not exist online
- Citation found online
- Citation not verifiable
- Cited claim not supported by source
- Cited claim partially supported or nuanced
- Cited claim supported by source
- Uncited claim not supported by online sources
- Uncited claim partially supported or nuanced
- Uncited claim supported by online sources

SPIN traces into C test code. Instead of replacing the whole pipeline at once, the project uses a bridge-based approach so that the Haskell version can be introduced gradually and checked against the original Python implementation. The port eventually moved away from line-by-line translation and towards a more Haskell-native design based on explicit state and smaller transformations. The two versions were then compared across the seven active models in the repository. This process found one real bug in the Haskell output, traced to template brace handling, which was fixed. After that, the remaining differences were limited to harmless formatting. Overall, the project shows that the refiner can be ported in a behaviour-preserving way.

Acknowledgments

Thank you Mum \& Dad.

Charles O'Malley

University of Dublin, Trinity College
April 2026

Contents

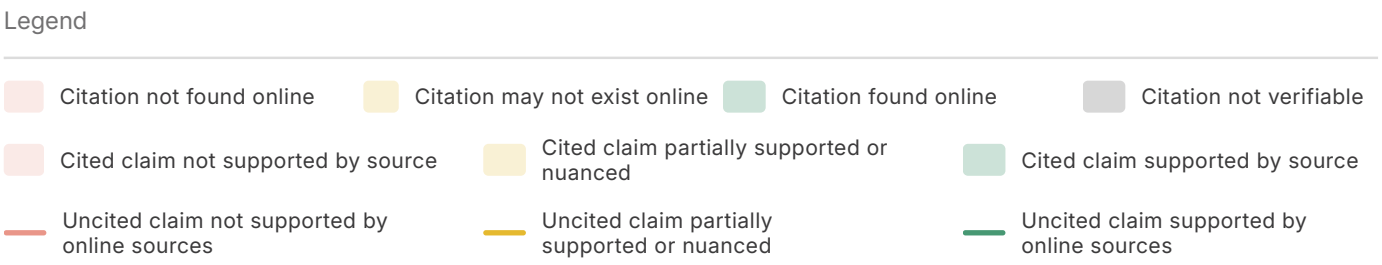
List of Tables

List of Figures

Chapter 1

Introduction

This chapter introduces the dissertation, motivates the decision to port part of the RTEMS test-generation pipeline from Python to Haskell, and explains the structure of the document. It first describes the technical context of the project, then states the aim and scope of the work, and finally shows how the remaining chapters support the overall argument.



1.1 Motivation

The repository used in this project contains a SPIN/Promela-based workflow for generating RTEMS validation tests. Promela models are explored with SPIN, the resulting annotated traces are refined into concrete test actions, and those actions are emitted as C test files. The stage studied here is the refinement step, implemented historically in Python. That step is small enough to replace in isolation, but central enough that any change to it can affect the entire pipeline.

The motivation for the project came from the structure of the existing refiner rather than from a failure of the generated tests. The Python version already generated useful RTEMS tests, but it accumulated mutable state, branching logic, and template expansion in one place. That made it harder to reason about the code, and it made larger structural changes risky. Haskell was therefore investigated not as a way to generate different tests, but as a way to re-express the same refinement logic with clearer state handling and stronger separation between stages.

In addition i was interested in exploring the coding paradigm that is fucntional programming, as i had nto had the oppurtunity to investigate it before due to the modules covering it not being avaible to me as a computer engineer. self teaching myself haskell would be a great way to explore it deeply.

1.2 Aim and Scope

The main aim of the dissertation is to determine whether the refine_command stage can be ported from Python to Haskell while preserving the behaviour of the existing generator. In practical terms, that means the Haskell version should accept the same annotated SPIN output and produce equivalent test code for the existing RTEMS models.

The scope of the work is deliberately narrow. The dissertation does not attempt to redesign the full testbuilder workflow, replace RTEMS itself, or prove that Haskell is universally better than Python for test generation. Instead, it focuses on one difficult, stateful part of the toolchain and studies how a gradual port can be carried out without losing confidence in the output.

In order to smoothly port the python program to haskell, the dissertation aims to create a incremental delegation strategy to keep end to end functionality consistent while the program is being created. the idea is to have an intermediary program that sits in between the file to be ported and the file that calls it. the original implementation will be preseved in situe with the new implementation. at the beggining the

intermediary program will give all work to the original implementation, but as the porting continues, the middleware will delegate more and more work to the new implementation until it is complete. i came up with this appraoch due to the desire to be entirely confident with the output of the system at all points in the project as i knew that the tests for rtems, the system which this codebase is for, are checked by rigourous quality control specialists. if i were to check the results of a partial program, i would want to be sure that it was progressing positively but i couldnt necessarily be sure of that just by looking at it myself. So i figured however that if the program in the end produced the same output as teh original that it would be a satisfactory port and i wouldnt have to know the details of the model checking or test generation to eb sure of it. so the aim to put this idea into practice

The main contributions of the dissertation are:

devising a bridge-based strategy for introducing large code changes to an existing codebase without interupting end to end functionality
an explicit-state Haskell implementation of the refinement stage;
an empirical comparison of Python and Haskell generated outputs across the seven active Promela models in the repository.

1.3 Dissertation Structure

Chapter 2 reviews the background needed for the project, including model-checker-based test generation, functional programming, and multilingual system design. Chapter 3 defines the problem more precisely and explains the design decisions behind the bridgebased port. Chapter 4 describes how the refiner was moved from Python to Haskell and where the implementation changed from line-by-line translation to a more Haskell-native design. Chapter 5 evaluates the result by comparing generated outputs and discussing the remaining limitations of the workflow. Chapter 6 closes the dissertation and identifies the most useful directions for further work.

Chapter 2

State of the Art

This chapter looks at the disserations place within the background work that made it project possible. It first reviews model checker based test generation, then discusses the functional programming ideas that motivate its use and the specific choice of Haskell, and finally considers the practical problems of

combining Python and Haskell inside one toolchain.

2.1 Model Checking and Test Generation

Model checking is normally presented as a verification technique, but in this project it is used in a more different way. SPIN explores the Promela model, produces counterexample traces. Traces are sequences of events, in this case recorded when assertions are violated, and those traces are then treated as the input for a test code generator rather than as the final result. The important point for this dissertation is that the generated test cases do not come directly from the model checker alone. They depend on a second translation stage that maps the traces onto RTEMS-oriented test code.
the pipeline is therefore a two stage system, which is incomplete with model checking alone. In the second step the traces produced by Spin are intepreted into test code using rtems specific refinement rules and C test code fragments and then assembled into a complete test file. the correctness of teh test produced therefore are dependent not just on the searching of the model checker but of the quality of the translation layer after it.

That distinction between the first and second stage of the test generation matters for this disseration because the dissertation does not replace the modelling or exploration stages. The Promela models, SPIN search, and RTEMS test structure already existed and remained untouched for this project. The difficult engineering problem of concern is the refinement step that comes after the promela mode and spin traces but before completed test code: reading annotation lines such as CALL, STATE, STRUCT, and SEQ,

tracking enough context to interpret them correctly, and emitting the right C fragments in the right order to create an executable that can verify teh behaviour of a real RTEMs deployment.
this translation layer is not simply a thin middleware that renames tokens with a fixed template where every line is intepreted separately to the whole. each line requires additional context from previous lines and must continue the context to the following lines. It must be able to determine whether the lines have entered or left higher level constructs such as strucutres or sequences. The translation layer can not be applied to any new system with manual adjustement first due to the fact that there are refinement dcitionaries and templates that are specific to each model that are required for the operation of the translation layer. For these reasons the refiner is better viewed as a stateful transformation system than a thin formatting layer. the work of Micskei and Majzik (Micskei and Majzik, 2006) comes closer to how the pipeline as seen in this dissertation exists. They use model checkers to create tests for embedded systems, showing that model checkers have value in actual systems and not just in theory as had been the case to that point. Ulrich

A broader example of the same general idea appears in the work of Zheng et al. (Zheng et al., 2007). They use model checking to derive tests for service behaviour described in BPEL-style models. The domain is different from RTEMS, and the modelling formalism is different from the Promela workflow used here, but the wider pattern is similar: model checking is used to systematically derive execution scenarios, and a second stage turns those scenarios into executable tests. That makes this dissertation easier to place in a broader test-generation tradition rather than treating it as an isolated RTEMS-specific toolchain.
et al. (Ulrich et al., 2010) are also relvant in that they emphasise the implementation step that comes after the model checker search. Ulrich wrote the program to create tests in Java but eventually came to decouple the backend from java in order to support other output langauges. This extensibility was something i wanted to emulate in this project

In the context of model checking and test generation for RTEMS, the output of a model checker test generator must be valid C code that fits the functionality and quality standards of the exisitn test suites generated by hand by the RTEMS team. this standard was achieved by my predecessor who worked on the whole of the test generator system as applied to RTEMS (Jaśkuć, 2022) applied model checker test generation to RTEMS in the first place and thus is extremely relevant to this dissertation as its the code that is actually being converted. The entire system is written in Python as it was mandated by authority in control of the repository. this led to the compromise that they programs were written in python depsite the developers desire to be writitg in something fucntional,

which ended up in the use of the external library Coconut which extends ptyhon to be more functional. why they would want to make it more fucntional is explored further in the next seciont

Seen in the light of this existing paper, this is not another demonstration that model checking can support automasted test generation, this has already been illustrated well by the paper. this dissertation instead is about whther the existing trace to code refinment system can be reimplemented without intterujpting or damaging the behaviour of the overall generation process. this changed focus makes the disseration closer to a study of a behaviour rpeserving codebase refactoring then to the work whos main contribution is the creation of a new modelling test generation framework. the model checking test generation is just the area of application that is being studied in this disseration.

2.2 Functional Programming and Haskell

A useful starting point for the Haskell side of the project is Hughes's argument that modularity is one of the strongest practical benefits of functional programming (Hughes, 1989). This is directly relevant here

because the refiner is a transformation: it reads one representation of behaviour and incrementally builds another. A style that makes intermediate state and data flow explicit is therefore attractive, especially when behaviour must be preserved while the internal structure changes.

That argument is useful here mainly because the refiner is full of small, state-dependent transformations. The code does not need a language chosen for theoretical but usually unrealised benefits; it needs a language whose normal style encourages clearer composition and more local reasoning. In that sense the appeal of functional programming in this dissertation is practical rather than abstract. The main benefit is not that the program becomes mathematically elegant, but that the steps by which one annotation line changes the next runtime state can be expressed more directly and thus more intuitively altered.

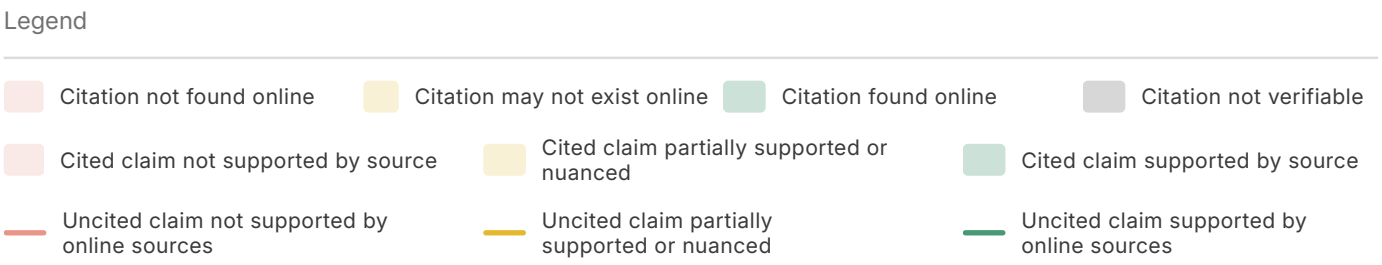
This project invovled learning about the history design goals of Haskell, summarised by Hudak et al. (Hudak et al., 2007). Haskell's combination of strong static typing, algebraic data types, pattern matching, and purity by default makes it well suited to programs that perform structured translation. THis is because, at least for this specific transformation there is constant pattern matching and also confusion that can arise from typing as traces which are ouputted as strings are necessarily processed, extracting the integers values held within. These properties do not automatically make the code correct , but they encourage an implementation style in which cases are more explicit and state changes are easier to trace.

That history is also relevant because it frames Haskell as a language designed to sup-

port reusable abstraction and a clear common standard, rather than as a niche experiment. For this dissertation that matters because the port is not trying to prove that Haskell is universally better than Python. It is simply using a language whose usual tools fit a translation problem well: explicit data modelling, case analysis, and a clearer separation between pure transformation and boundary-facing input and output.

It is also worth talking about the scope of haskells utilisation. The haskell as implemented in this project was learned for the purpose of this dissertation rather than from an extended period of getting to sit with the language in a formal academic module. This matters in that the port being done by the author will not be able to fully use all the theoretical benefits of the language to its fullest extent. Even so, with just a subset of the languages potential, the refinement program can be improved and then it will greatly ease further contributors in maximising the programs potential

2.3 Multi-Language Systems



The project is also part of a broader class of multilingual systems, where one language is introduced into an existing codebase rather than replacing it completely. A recent empirical study by Yang et al. (Yang et al., 2024) highlights interface boundaries, data conversion, and error diagnosis as recurring sources of friction. Those concerns match the experience of this project closely. The main difficulty was not writing isolated Haskell functions, but deciding how Python and Haskell should communicate without making the combined system more fragile than the original one.

This problem shaped the eventual architecture. Instead of a deep foreign-function interface, the project used a narrow subprocess protocol over standard input and output. That choice keeps the boundary explicit. Configuration is passed in as JSON, annotation lines are refined one at a time, and the rendered test body is returned at the end. The approach is less elegant than a direct import, but it is easier to inspect, easier to debug, and better aligned with the existing Python pipeline. It also reflects the main lesson of the multilingual-systems literature in a fairly direct way: once the boundary is explicit, it becomes easier to reason about what data crosses it and where failures should be diagnosed.
the later design and implementation chapters unpack this in more concrete terms by lookijg at the trade offs in inspectability, debugabillity and API stability.

Two concrete integration options were considered during the project. The HaPy library (Fisher, 2014) suggested a more direct Python-to-Haskell interface by exposing compiled Haskell code to Python, but its documented setup depends on shared-library builds and a limited set of supported exported types. The call-haskell-from-anything li-

brary (Hambüchen, 2011) explored a more general MessagePack-based boundary over the C foreign-function interface. Both were useful as engineering reference points, but neither was adopted. The repository needed a controlled way to preserve the existing Python API while changing the implementation behind it, and the simpler subprocess boundary turned out to be the safer fit for that requirement.

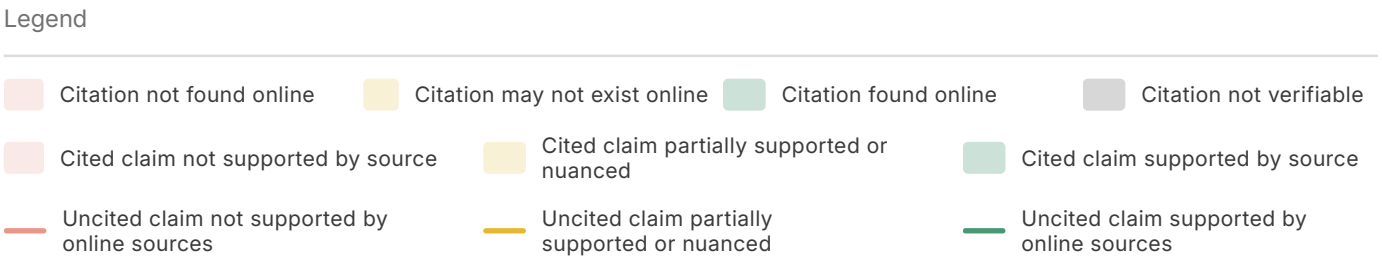
2.4 Closely Related Work

The most directly relevant prior work is the earlier dissertation by Jaśkuć (Jaśkuć, 2022). That work is important because it establishes the value of the underlying RTEMS/SPIN framework itself. The present dissertation does not need to justify that the entire modelling approach is worthwhile; it builds on an existing system that had already shown that Promela models and SPIN traces could be refined into useful RTEMS tests.

The difference in emphasis is therefore clear. The earlier work extended the range and usefulness of the RTEMS modelling framework. This dissertation instead studies the maintainability of one important part of that framework by replacing the refinement stage with a Haskell version. The contribution is not a new manager model or a new coverage result, but a controlled port of the translation logic used by the existing models.

Source	Main Idea	Relevance Here
Why Functional Programming Matters (Hughes, 1989)	Functional programming improves modularity and composition	Motivates the move from implicit mutation toward explicit transformations
A History of Haskell (Hudak et al., 2007)	Explains the design goals of Haskell as a common non-strict functional language	Justifies Haskell as a plausible replacement language for a structured translation stage
Yang et al. (Yang et al., 2024)	Identifies interface boundaries and data conversion as common developer difficulties	Motivates the explicit subprocess bridge between Python and Haskell
Fraser et al. (Fraser et al., 2009)	Surveys how model checkers are used to derive test cases from counterexamples and witnesses	Clarifies that this dissertation focuses on the later refinement step that makes those traces usable
Micskei and Majzik (Micskei and Majzik, 2006)	Generates tests for event-driven embedded systems using model checkers	Shows that model-checkerdriven test generation is relevant in embedded, reactive software settings
Ulrich et al. (Ulrich et al., 2010)	Moves from formal scenarios to concrete test implementations via Promela	Reinforces that a non-trivial implementation step exists between formal scenarios and runnable tests
Zheng et al. (Zheng et al., 2007)	Uses model checking to derive executable tests from behavioural models	Shows that turning modelchecker output into tests is a broader pattern, not just an RTEMS-specific one
Staroletov (Staroletov, 2020)	Models partitioned real-time operatingsystem behaviour in Promela for SPINbased verification	Provides systems-software context for using Promela and SPIN on operatingsystem behaviour
Jaśkuć (Jaśkuć, 2022)	Demonstrates the value of the RTEMS/SPIN testgeneration framework	Provides the immediate technical context that this dissertation extends

In summary, the literature and engineering context suggest two main conclusions. First, the project looks



to be a study of how model-checker output can be preserved while the refinement stage is restructured. Second, the challenge is not only one of programming language choice, but also one of managing a clear interface between old and new implementations.

Chapter 3

Design

This chapter turns the background of the previous chapter into concrete design decisions. It defines the specific problem within the existing RTEMS toolchain, identifies the constraints that any replacement must satisfy, and explains the bridge-based design used to introduce Haskell gradually rather than by one large swap.

3.1 Problem Formulation

The problem addressed by this dissertation is not that the existing generator failed to produce useful RTEMS tests. The problem is that one central stage of the pipeline, the Python refiner, became difficult to restructure safely as the project grew. The testbuilder workflow already had working Promela models, a working SPIN-based exploration stage, and working C test generation. Changing the refiner therefore meant changing the part of the system where the risk of regression was highest.

The existing Python refiner combines several kinds of work in one place. It parses annotation lines, tracks context such as whether refinement is currently inside a STRUCT or SEQ, performs lookups in a refinement dictionary, and accumulates generated code fragments. This arrangement works, but it makes it harder to reason locally about how one line of input affects the next state of the translation.

Three properties of the original implementation were especially important. First, context is stored as mutable object state. Second, the main refinement function handles many annotation forms in one place. Third, template expansion and code accumulation are interleaved with control flow. Each of these is manageable in isolation, but together they make structural changes riskier than they need to be.

3.1.1 Identified Challenges

The first challenge was behavioural preservation. The repository already contained working models and a working Python refiner. A replacement could only be justified if it produced the same observable output

for the existing model set.

The second challenge was integration. The refiner is not a standalone tool. It is called from the existing Python pipeline and depends on YAML refinement dictionaries, language templates, and per-model header fragments. Any design therefore had to fit around those existing inputs rather than invent a new workflow.

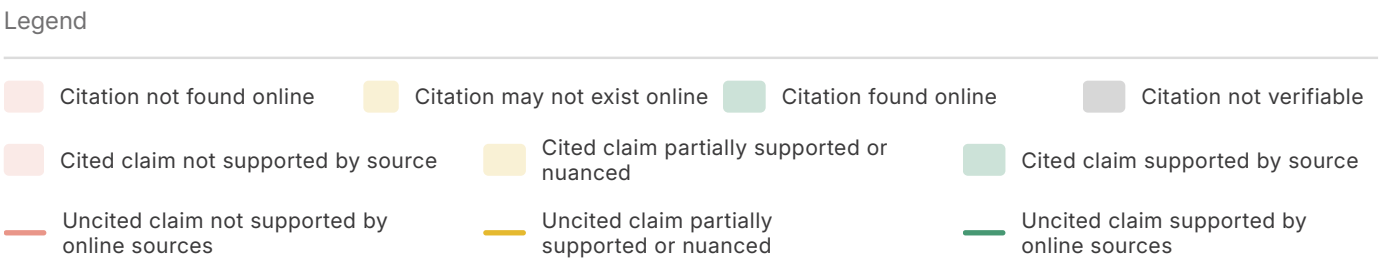
The third challenge was scope control. Replacing the entire pipeline at once would have made it difficult to distinguish language-porting issues from unrelated tooling issues. A narrower design was needed so that the hardest stateful component could be rewritten first and checked continuously against the original behaviour.

3.1.2 Proposed Work

The design adopted in this dissertation is a staged port of refine_command. The Haskell implementation keeps the same external role as the Python one: it receives tokenised annotation lines, updates its refinement state, and eventually renders a test body. The surrounding Python code is retained so that existing configuration files, model assets, and generation scripts can continue to drive the process.

The central design decision is therefore not only to use Haskell, but to introduce it behind a bridge that preserves the historical Python-facing interface. That keeps the migration incremental. The new implementation can be checked with the same inputs as the old one, and output differences can be detected before they spread into the rest of the workflow.
the bridge was placed at the boundary between spin2test.py and refinecommand.py for good reason. when spin2test.py hands work to refinecommand.py, most of the surrounding setup has already been done, the correct RTEMS model files have been found, the .spn ouput has been split into individual spin trace lines nad the YAML refinement dictionaries specific to the RTEMS model have already been loaded. This leaves refinecommand.py with a more singular and specialised job. it only has to recieve configuration settings at the beginning, process the spin trace lines one by one, and then return the completed test scripts. putthing the bridge here isolates the stateful, pattern matching part of the pipeline while keeping the function of the ported file more consolidated by not also having it handle other parts of the workflow. this design decision complements the decision to use the bridge to gradually port refinecommand.py. this is because when comparing the output of the new file with the original, if theres a difference

then i can be more sure that it has to do with the test conversion pipeline and not with some auxiliary helper function.



Two more aggressive and involved alternatives were considered for this bridge program but were ultimately not chosen. Firstly a full rewrite at the outset would have moved the bridge location outwards, meaning that more of the system could be interfaced with using haskell, but it would also have linked refine_command.py changes with spin2test.py, file orchestration and template loading. as mentioned before this would have made regressions trickier to pinpoint, having a wider area to check.

Secondly, a Foreign Function Interface (FFI) is another appraoch to the mediating between the haskell and non haskell portions of the code, this time having haskell be called as a compiled library from haskell, like the equivalent of a native library. this way keeps the refinment inside of one process, as opposed to have the bridge process and the refinement sub process. It wasnt chosen however as it creates a more brittle boundary where things like function signatures, data layout and memory ownership have to be decided exactly. additonally it makes debugging less transparent as the haskell function is called directly by the python code. the stdin/out message pipeline contrasts this by being quite flexible with its command response system and being extremely visible for debugging, as every message is published to console. these reasons explain why this appraoch was chosen

3.2 Overview of the Design

The overall design separates the system into three logical parts: SPIN produces annotated traces from Promela models, the bridge passes those annotations and configuration data to the Haskell refiner, and the existing Python-side tooling assembles the final test files around the rendered body. This decomposition isolates the port to the refinement stage while leaving the rest of the pipeline stable.

In the repository as it stands now the direction of control remains Python centric. spin2test.py reads the generated Spin trace file, keeps only lines that start with @@@, splits those lines into pieces for the test code templates, loads the refinement dictionary from YAML, and constructs the command object. That object name is preserved so the programs that call for it do not have to change, but the implementation behind it is now the bridge in combinedhaskellbridge.py. The bridge starts the Haskell executable in interactive mode and copies the methods that the existing Python pipeline expects, including setupLanguage, refineSPINLinemain, and testbody.

A line-oriented protocol was used between Python and Haskell because it made the interface explicit and easy to inspect. The bridge sends an initial JSON configuration,

then a sequence of refinement commands, and finally requests the rendered test body. The protocol is simple enough to log and replay, which proved useful during debugging.

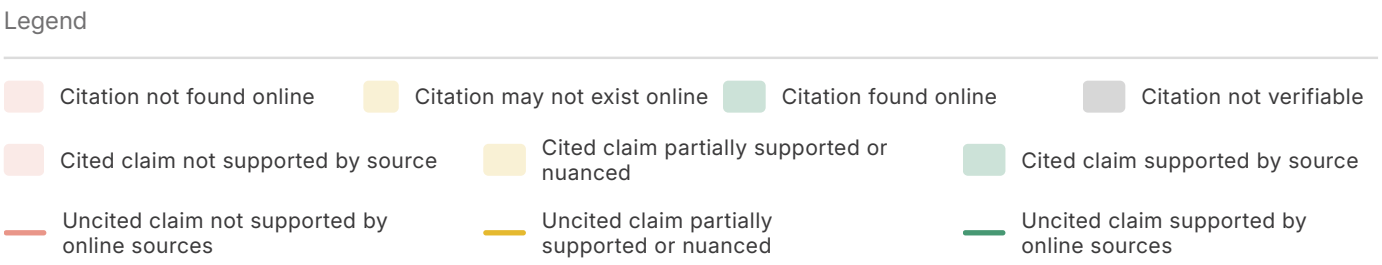
```
INIT JSON:{...}
RESPONSE:SUCCESS:INIT JSON received
COMMAND:setupLanguage
RESPONSE:SUCCESS:setupLanguage
COMMAND:refineSPINLine:["4", "CALL", "WaitForSuspend", "2", "resumeRC"]
RESPONSE:SUCCESS:refineSPINLine
COMMAND:testBody
RESPONSE:SUCCESS:testBody:BEGIN
...
RESPONSE:SUCCESS:testBody:END
```

Listing 3.1: Simplified bridge exchange between Python and Haskell

Here is a concrete example to help show why this appraoch works. In the task-manager model, spin2test.py can encounter the spin line ["4", "CALL", "WaitForSuspend", "2", "resumeRC"] while scanning the generated trace. Python does not interpret that line beyond tokenising it, i.e. reducing every part of the trace line into a separate piece. The token array is sent over the bridge, and the Haskell refiner then performs the stateful work. It begins by recording process ID 4, then applying any switch or annotation handling required by the runtime flags, in this case its "CALL" which doesnt require any switching, then looking up WaitForSuspend in the refinement dictionary, and formatting the associated template with the arguments 2 and resumeRC.

```
WaitForSuspend: |
 Tlog( TNORMAL, "Waiting for Task(%d) to Suspend", {0}, {1} );
 do {{
{1} = ( *ctx->t_isSuspend )( {0} ? taskID[{0}] : 0xffffffff );
 rtemstaskwake_after( 100 );
}} while ({1} != RTEMSALREADYSUSPENDED);
```

Listing 3.2: Refinement template for WaitForSuspend
here is the template that is called up by searching for "WaitForSuspend" in the refinement dictionaries. the spots where theres number are where the tokens are slotted in



```
Tlog( TNORMAL, "Waiting for Task(%d) to Suspend", 2, resumeRC );
do {
 resumeRC = ( *ctx->t_isSuspend )( 2 ? taskID[2] : 0xffffffff );
 rtemstaskwake_after( 100 );
} while (resumeRC != RTEMSALREADYSUSPENDED);
```

Listing 3.3: Generated C after template substitution
and here you can see what it looks like when the values from the spin line are subsituted in.

The important design point is that Python does not need to know how that fragment is built; it only preserves the boundary and passes the same observable inputs that the old refiner used.

The state after that command remains inside the Haskell process. The generated lines are accumulated in memory as part of the runtime state rather than being written to disk immediately. Only when Python later calls test_body() does the bridge send COMMAND:testBody, receive the text body between BEGIN and END markers, and then assemble the final output file. It does this by writing the predefined preamble and postamble text, the rendered body, and the run fragment. This separation is why the bridge can stay narrow: Python keeps responsibility for file-system orchestration, while Haskell owns the stateful translation from token array stream to generated C.

JSON plus line-oriented commands were sufficient because only two payload shapes had to cross the boundary: one configuration object at startup and one token array per annotation line. Everything else is a fixed command or a return of the body text. A richer RPC framework would have added complexity without improving the one property the dissertation needed most, namely a boundary that could be logged, replayed, and compared directly against the historical Python behaviour.

This design also made differential checking a key focus of the project rather than an afterthought. Because the Haskell refiner could be exercised with the same traces as the Python original, text diffs on generated tests became the main feedback loop. That in turn made it possible to move from a line-by-line translation strategy to a more Haskell-native design without losing confidence in the observable output.

Against the two main alternatives, the subprocess bridge performed best on the criteria that mattered most for this dissertation. It preserved the historical Python API because callers still interacted with a

familiar command-style object. It was easier to debug because every request and response could be inspected as plain text. And it supported debugging checking because the same annotation traces could be replayed through both implementations with minimal surrounding change. Its main cost was explicit message passing, but that cost was acceptable because the boundary was intentionally narrow.

3.3 Summary

In summary, the design of the dissertation is conservative at the system boundary and experimental inside the refiner itself. The boundary stays stable so that behaviour can be checked continuously, while the implementation behind that boundary is free to adopt a clearer Haskell structure.

Chapter 4

Implementation

This chapter describes how the design of the previous chapter was realised in the repository. It concentrates on mechanics rather than motivation: how the Python-facing workflow was preserved, how the bridge starts and talks to the Haskell executable, how diff-based comparison exposed regressions, and how individual annotation lines become generated C.

4.1 Overview of the Solution

the implementation followed a gradual transition from the original Python refiner to a Haskell refiner while keeping the rest of the pipeline recognisable. In this repository snapshot, the main artefacts were the earlier Python refiner purepythonrefinecommand.py, spin2test.py, combinedhaskellbridge.py, and the new refinecommand.hs. The objective was not to replace everything at once, but to replace the refinement stage in a way that could be checked end to end against the old behaviour.

A full rewrite from the start would have been risky. The refiner is small enough to look manageable, but it contains many state-dependent branches and many points where template expansion affects the final test file. Reaching the end of a full rewrite before comparing outputs would therefore have made it difficult to localise errors. The staged approach reduced that risk by keeping differential comparison in the loop throughout the port.

The testing and debugging effort was directed toward functional equivalence with the Python generator.

The Python output already matched the expected RTEMS test structure, so the main implementation question was whether the Haskell version could preserve that observable behaviour while using a clearer internal structure.

The conservative part of the implementation was therefore the system boundary. spin2test.py would still discover the generated trace, load the YAML refinement dictio-

nary, and write the final test file around the rendered body. The experimental part was the refiner behind that boundary. This split made it possible to change the most stateful component first while keeping the surrounding workflow stable enough to compare outputs continuously.

4.2 The Implementation

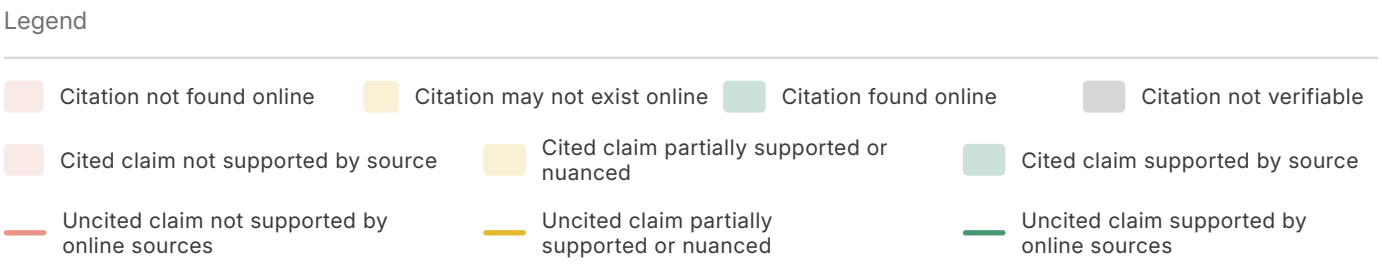
4.2.1 The Bridge

During the transition, a bridge layer preserved the historical Python-facing command interface while moving the refinement work behind a subprocess. Python remained responsible for orchestrating the wider pipeline, but Haskell could now be exercised inside that pipeline without forcing an immediate rewrite of every caller.

At the Python entry point, the actual calling pattern changed less than the architectural shift might suggest. spin2test.py still reads the generated .spn file, filters lines beginning with @@@, tokenises those lines, loads the refinement YAML, and writes the final output by placing generated code between the existing preamble, postamble, and run fragments. The main change is that it now imports combinedhaskellbridge.py as refine and constructs a bridge-backed command object instead of just importing refine_command.py

```
cmd = refine.command(ref_dict)
cmd.setupLanguage()
for ln in annote_bundle:
 cmd.refineSPINLine_main(ln)
for line in cmd.test_body():
 tstfile.write(line)
```

Listing 4.1: Python-side calling preserved by the bridge



Inside combinedhaskellbridge.py, the public command wrapper delegates to a smaller _HaskellBackend. Startup has three visible steps. First, Python locates the refine-haskell executable and launches it in interactive mode with standard input and output. Second, it builds one JSON object containing the refinement dictionary and the three runtime flags outputLOG, annoteComments, and outputSWITCH. These are options that can be turned on and off, outputLOG for examplle toggles whether a log file is created. Third, it sends that object as an INIT JSON: . . . line and waits for a success response before continuing. This makes configuration loading explicit and gives one early failure point if JSON parsing or language setup goes wrong.

Communication after startup is carried entirely through standard input and output. The bridge sends fixed commands such as COMMAND:setupLanguage and COMMAND:testBody, and it sends spin lines as JSON arrays via COMMAND:refineSPINLine: . . . On the Haskell side, handleLine decodes the command prefix, updates the in-memory runtime state, and responds with either RESPONSE: SUCCESS: . . . or RESPONSE:FAILED: . . . On the Python side, expect() checks that each response begins with the required prefix. If readline() sees an unexpected EOF it triggers an immediate failure. The result is a deliberately plain protocol in which invalid commands, malformed JSON, or a crashed subprocess are surfaced quickly at the boundary.

INIT JSON:{"ref_dict": {...}, "flags": {...}}
RESPONSE:SUCCESS:INIT JSON received
COMMAND:setupLanguage
RESPONSE:SUCCESS:setupLanguage
COMMAND:refineSPINLine:["4", "CALL", "WaitForSuspend", "2", "resumeRC"]
RESPONSE:SUCCESS:refineSPINLine
COMMAND:testBody
RESPONSE:SUCCESS:testBody:BEGIN
...
RESPONSE:SUCCESS:testBody:END

Listing 4.2: Short request-response exchange used during refinement
After successful initialisation, the Haskell executable moves into a simple command loop that continues until standard input reaches EOF. In practice, the process lifetime is therefore tied to the Python caller. Once spin2test.py has requested the final test lines and has exited after writing the generated test file, the the standard input/ output close and the interactive loop terminates. This keeps the protocol small, at the cost of relying on EOF rather than explicit session finalisation.

This arrangement is deliberately plain, but that simplicity was useful in practice. Python continued to own file-system orchestration and the historical command interface, while Haskell owned only the translation from spin line stream to emitted body text. That narrow split is what allowed the bridge to be inserted with relatively little change on the Python side.

4.2.2 The Diff Script

A constant part of the porting process was checking the generated test files against the ones produced by the original system. At first this was done manually with an online text comparison tool, but that approach became unworkable once a single change could

affect many generated files. A small local diff script therefore became part of the working process.

```
--- diff for tr-task-mgr-model-16.c ---
--- /home/a/Documents/Python_gen/tr-task-mgr-model-16.c
+++ /home/a/Downloads/rtems/RTEMS-SMP-Formal-main/formal/promela/models/task-
 mgr/gen/tr-task-mgr-model-16.c
@@ -177,10 +177,10 @@
 Tlog(TNORMAL,"@@@ 4 CALL WaitForSuspend 2 resumeRC");
 Tlog( TNORMAL, "Waiting for Task(%d) to Suspend", 2, resumeRC );
do {
do {{
 resumeRC = ( *ctx->t_isSuspend )( 2 ? taskID[2] : 0xffffffff );
 rtemstaskwake_after( 100 );
} while (resumeRC != RTEMSALREADYSUSPENDED);
}} while (resumeRC != RTEMSALREADYSUSPENDED);
/* SWITCH[2] Suspension of proc4 in favour of proc5 */
/* SWITCH[3] ReleaseTestSyncSema of proc4 (sometime) after proc5 */
@@ -370,6 +370,7 @@
}
```

Listing 4.3: Example output from the diff workflow showing a generated file diverging from the reference output. In practice, even a single extra line was worth investigating because small formatting mismatches could indicate a deeper error in template expansion.

The diff-based approach was intentionally strict. It reported trivial whitespace differences as well as genuinely important mismatches. That occasionally produced noise, but it was still useful because it made it harder for a small divergence to go unnoticed and later turn into a larger behavioural discrepancy. In practice, most debugging started from a diff, then moved backwards through the relevant refinement dictionary entry and the code path that emitted it.

The brace mismatch shown in the diff output became a clear example of why this workflow was useful. In the task-manager model, the Haskell output initially contained do \{\{ and \}\} while where the Python generator produced a single pair of braces, i.e. the valid C syntax. The underlying problem was not in the control-flow logic of the generated test, but in template handling. The refinement dictionaries store literal braces doubled so that Python str.format() will leave them intact. The first Haskell version replaced

numbered placeholders but did not collapse doubled braces back to single ones afterwards. The result was syntactically invalid generated C even though the surrounding structure was correct. Fixing the haskell refiner ended up being as simple as adding an addtional case in teh pattern matching to account for it which restored parity with the Python output. This was a good example of a bug that was easiest to see at the level of emitted files rather than at the level of isolated helper functions.

Smaller tests were also useful around configuration handling and local translation behaviour, but they were less decisive than the end-to- end diff checks. The hardest bugs were usually interactions between several moving parts at once: lines, templates, states, and rendering. For those cases, comparing the final generated C remained the most informative test.

4.2.3 Refine Command

This was the main file to be converted. Its job is to take annotated SPIN output and refine it into concrete test code by matching annotation lines against entries in a model-specific refinement dictionary. Those entries are small templates of C code with placeholders for names, values, and identifiers taken from the trace.

At the beginning of the port, the Python code was translated into Haskell as closely as possible. The intention was to keep behaviour stable by keeping the structure stable. What became clear quite quickly, however, was that this strategy made the Haskell version look like Python written in another syntax. It preserved the awkward parts of the original without gaining much clarity from the new language.

The turning point came in state handling. The program has to remember whether the current annotation is inside a structure, inside a sequence, or part of ordinary top-level output. The Python version manages this with mutable object fields that are updated as each line is processed. That behaviour can be copied in Haskell, but copying it directly would have reproduced the same difficulty of following how state changes over time.

The more Haskell-native approach was to make the state explicit. Instead of mutating fields on one long-lived object, each stage receives a runtime state, transforms it, and returns the next one. At that point the port stopped being a line-by-line translation and became a Haskell implementation that happened to preserve the same outputs. The aim from then on was not structural similarity to the Python source, but behavioural equivalence of the generated tests.

The refiner therefore ended up with a clear two-level structure. One level applies generic steps that every annotation line goes through, such as PID collection, optional switch insertion, and optional annotation logging. The second level handles the opcode-

specific work, such as expanding a CALL template or entering STRUCT/SEQ mode. This separation made it easier to reason about what was common to all lines and what was specific to one kind of annotation.

This change is visible in the following snippets.

Python state stored and mutated on the object

```
self.inSTRUCT = False
self.inSEQ = False
self.seqForSEQ = ""
self.inName = ""
case [pid, "SEQ", name]:
 self.inSEQ = True
 self.seqForSEQ = ""
 self.inName = name
case [pid, "SCALAR", "_", value]:
 self.seqForSEQ = self.seqForSEQ + " " + value
```

Listing 4.4: Python state stored and mutated on the object

In the Python version the state lives on the command object. As each SPIN line is read these values are updated in place, and later branches depend on whatever earlier branches left behind.

Haskell state made explicit
data RuntimeState = RuntimeState
{ rtInSTRUCT :: Bool
, rtInSEQ :: Bool
, rtSeqForSEQ :: Text
, rtInName :: Text
, rtDefCode :: [Text]
, rtDeclCode :: [Text]
, rtTestCodes :: [(Int, [Text])]
, rtCurrentId :: Int
, rtSwitchNo :: Int
}

Listing 4.5: Explicit runtime state in the Haskell refiner
The same information is still present, but it is now grouped into one explicit record. Rather than hidden mutation on an object, each function takes a state and returns the next one.

One SPIN line producing the next state

refineSPINLineMain :: [Text] -> RuntimeState -> RuntimeState
refineSPINLineMain ln rt0 =
 let rt1 = collectPIdsTokens ln rt0
 rt2 = if outputSWITCH (rtFlags rt1) then switchIfRequired ln rt1 else rt1
 rt3 = if annoteComments (rtFlags rt2) then logSPINLine ln rt2 else rt2
 in refineSPINLine ln rt3

Listing 4.6: Processing one SPIN line by returning a new state
This is the clearest expression of the change in approach. Instead of one long-lived object being updated in place, each stage produces a fresh version of the runtime state before the next stage runs.

The split between refineSPINLineMain and refineSPINLine also matters conceptually. The first function handles generic pre-processing that can apply to many opcodes. The second handles the meaning of the

opcode itself. For a CALL annotation, for example, switch insertion and annotation logging are decided before the dictionary lookup for the named template happens.

CALL refinement via dictionary lookup

```
pidTxt:"CALL":name:args →
 case lookupRef name rt of
 Nothing → addCode (pidFromText pidTxt) [rtEOLC rt  " CALL: no refinement
 entry for '"  name  "'"] rt
 Just code →
 let callCode = if null args then code else formatTemplate code args
 in addCode (pidFromText pidTxt) (splitLines callCode) rt
```

Listing 4.7: CALL annotations refined by template lookup

This is one place where the Haskell version became more direct than a literal translation. The branch says almost exactly what the operation means: find the template, instantiate it with the supplied arguments, split it into output lines, and append those lines to the runtime state under the current PID.

Sequence handling as an explicit transition

```
_pidTxt:"SEQ":name:[] →
 rt { rtInSEQ = True, rtSeqForSEQ = "", rtInName = name }
 pidTxt:"SCALAR":"":value:[] →
 rt { rtSeqForSEQ = rtSeqForSEQ rt  " "  value }
```

Listing 4.8: Sequence state handled as an explicit transition

Sequence handling was where this change felt most useful. In this context a sequence is an ordered run of values collected between SEQ and END and later emitted through a matching _SEQ template. In the Haskell version, entering that mode and accumulating those values are both visible directly in the returned state.

The same explicit-state style also makes it easier to explain one full translation example. In the task-manager model, the refiner may see an earlier declaration line for resumeRC and then later a CALL annotation that uses the same name inside a generated polling loop.

```
@@@ 4 DECL byte resumeRC 0
```

```
@@@ 4 CALL WaitForSuspend 2 resumeRC
WaitForSuspend: |
 Tlog( TNORMAL, "Waiting for Task(%d) to Suspend", {0}, {1} );
 do {{
{1} = ( *ctx->t_isSuspend )( {0} ? taskID[{0}] : 0xffffffff );
 rtemstaskwake_after( 100 );
}} while ({1} != RTEMSALREADYSUSPENDED);
```

Listing 4.9: Annotation and refinement template used in the taskmanager example

The declaration line matters because the later CALL template does not use resumeRC as a literal value. It uses it as the name of a C variable that must already exist in the generated test. When the CALL line is processed, the runtime state is updated in stages. First, the PID is recorded in rtProcIds. Second, if switch output is enabled and the active PID changes, the bridge can emit the corresponding SUSPEND and WAKEUP template expansions. Third, if annotation comments are enabled, the raw annotation line is logged into the generated output. Only after those generic steps does the CALL branch look up WaitFor

Citation check (1)

0 don't exist online | 0 may not exist online | 1 exist online | 0 are not verifiable

Why Functional Programming Matters

Source was found online.

We are confident that we matched the source.

Most similar source found

J. Hughes. "Why Functional Programming Matters". The Computer Journal. https://academic.oup.com/comjnl/article-abstract/32/2/98/543535. Accessed 2026-04-12T16:24:52.812804

Title: Match Author: Match Date: Match URL/DOI: Missing Publisher: Missing

The citation exists: we found a source that matches it closely.

Cited claims (19)

0 not supported by source | 0 partially supported or nuanced | 2 are supported by source

- "the work of Micskei and Majzik (Micskei and Majzik, 2006) comes closer to how the pipeline as seen in this dissertation ..."

  Source stance is unknown.

- "This project invovled learning about the history design goals of Haskell, summarised by Hudak et al. (Hudak et al., 2007..."

  Source stance is unknown.

- "A recent empirical study by Yang et al. (Yang et al., 2024) highlights interface boundaries, data conversion, and error ..."

  Source stance is unknown.

- "A broader example of the same general idea appears in the work of Zheng et al. (Zheng et al., 2007)."

  Source stance is unknown.

- "et al. (Ulrich et al., 2010) are also relvant in that they emphasise the implementation step that comes after the model ..."

  Source stance is unknown.

- "this standard was achieved by my predecessor who worked on the whole of the test generator system as applied to RTEMS (J..."

  Source stance is unknown.

- "A useful starting point for the Haskell side of the project is Hughes's argument that modularity is one of the strongest..."

  Claim is strongly supported

  Why the Source supports the Claim: The Source (Hughes, 1989) states that "modularity is the key to successful programming" and functional languages "contribute greatly to modularity", directly supporting the Claim.

- "The HaPy library (Fisher, 2014) suggested a more direct Python-to-Haskell interface by exposing compiled Haskell code to..."

  Source stance is unknown.

"The call-haskell-from-anything li-

brary (Hambüchen, 2011) explored a more general MessagePack-based boundary over the ..."

Source stance is unknown.

"The most directly relevant prior work is the earlier dissertation by Jaśkuć (Jaśkuć, 2022)."

Source stance is unknown.

"| Why Functional Programming Matters (Hughes, 1989) | Functional programming improves modularity and composition | Motiv..."

Claim is strongly supported

Why the Source partial support to Claim: Source states "functional languages push those limits back" on modularity, aligning with Claim's "improves modularity and composition".

"A History of Haskell (Hudak et al., 2007)"

Source stance is unknown.

"Yang et al. (Yang et al., 2024)"

Source stance is unknown.

"Fraser et al. (Fraser et al., 2009)"

Source stance is unknown.

"Micskei and Majzik (Micskei and Majzik, 2006)"

Source stance is unknown.

"Ulrich et al. (Ulrich et al., 2010)"

Source stance is unknown.

"Zheng et al. (Zheng et al., 2007)"

Source stance is unknown.

"Staroletov (Staroletov, 2020)"

Source stance is unknown.

"Jaśkuć (Jaśkuć, 2022)"

Source stance is unknown.

Uncited claims (3)

0 not supported by online sources | 2 partially supported or nuanced | 1 are supported by online sources

"The process of comparing the Haskell version with the original Python implementation found one real bug in the Haskell o..."

"The earlier dissertation by Jaśkuć extended the range and usefulness of the RTEMS modelling framework."

> Supports claim
> "Abstract".
> https://www.scss.tcd.ie/publications/theses/diss/2022/TCD-SCSS-DISSERTATION-2022-016-ABSTRACT.pd
> Accessed 2026-04-12T16:24:57.916035

The abstract notes the dissertation “overhauled, extended and applied” the existing RTEMS modelling framework to a new component (Barrier Manager) and produced a “cleaner and more extendable” framework, directly showing it broadened the framework’s range and usefulness, thus supporting the claim.

"The line number reference is @@ -177,10 +177,10 @@."

Nuanced
"Diff - a4830a7a4564b78cb0196b931bd4eea918dd1973^2 ...".
https://chromium.googlesource.com/external/github.com/git/git/+/a4830a7a4564b78cb0196b931bd4eea91
Accessed 2026-04-12T16:24:52.602678

The source is a collection of unrelated diff hunks that never mentions the exact line-range "@@ -177,10 +177,10 @@"; it merely shows other @@ headers. Since it neither confirms nor denies that specific reference, the source's stance toward the claim is neutral.

Nuanced
"linux-yocto/4.12: update to v4.12.26 - poky".
https://git.yoctoproject.org/poky/commit/?id=89b94e40f4f7479da65e6fd74c546e9d448e9b65.
Accessed 2026-04-12T16:24:53.767453

The source shows diff hunks such as "@@ -11,13 +11,13 @@" and other line-range markers, but it never mentions the specific reference "@@ -177,10 +177,10 @@". Since it provides no support or opposition to that exact line reference, its stance is neutral.

Nuanced
"Diff - refs/heads/stable-2.4^2..refs/ ...".
https://gerrit.googlesource.com/gerrit/+/refs/heads/stable-2.4%5E2..refs/heads/stable-2.4/.
Accessed 2026-04-12T16:24:52.970816

The source lists several diff hunks (e.g., @@ -1,5 +1,6 @@, @@ -15,7 +15,7 @@) but never includes the specific line reference @@ -177,10 +177,10 @@. Since it gives no support or opposition, its stance toward the claim is neutral.

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 17 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scene Writing Reports as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our Writing Reports through Google Docs). Check out our list of several recommendations on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.